

Gadget IVR: Python İle Telekom & Medya Akış Rehberi

Bu rehber; Node-RED üzerinde JavaScript yerine **Saf Python (Python 3)** kullanarak **Piper TTS**, **FFmpeg** ve **Baresip** sistemlerini nasıl orkestre edeceğinizi adım adım anlatır.

Node-RED ortamımızda `python3` ve `python-function` düğümü yüklüdür.

1. PIPER TTS (Nöral Metinden Sese Dönüştürme)

A. Arka Planda Ne Oluyor? (Low-Level Mekanizma)

Piper TTS servisi arka planda Python (FastAPI) kullanır ve 5000 (Host üzerinde 5005) portundan HTTP REST API sunar. - **İstek (HTTP POST)**: `http://127.0.0.1:5000/api/tts` (Lokal dışarıdan `http://localhost:5005/api/tts`) - **JSON Body**: `json { "text": "Merhaba Gadget IVR sistemine hoş geldiniz.", "model": "tr_TR-eren-medium", "filename": "welcome.wav" }` - **Fiziksel Çıktı**: Piper container'ı sesi üretir ve ortak volume olan `/tmp/media/welcome.wav` (Host makinede `./data/media/welcome.wav`) olarak kaydeder. - **Yanıt (JSON Response)**: `json { "status": "success", "file_path": "/tmp/media/welcome.wav", "filename": "welcome.wav", "model_used": "tr_TR-fahrettin-medium", "text": "Merhaba..." }`

B. Node-RED Python Yapılandırması

Senaryo 1: Bağımsız Statik Ses Dosyası Üretimi (Anons Kaydetme)

1. **Inject Node (Tetikleyici)**:
2. **Payload Type**: `JSON`
3. **Payload Value**: `json { "text": "Sayın müşterimiz, faturanızın son ödeme tarihi gelmiştir.", "model": "tr_TR-eren-medium", "filename": "fatura_anons.wav" }`
4. **HTTP Request Node** :
5. **Method**: `POST`
6. **URL**: `http://127.0.0.1:5000/api/tts`
7. **Return**: `a parsed JSON object`
8. **Debug Node** :
9. **Output**: `msg.payload` (Üretilen dosya yolunu `/tmp/media/fatura_anons.wav` ekrana basar).

Senaryo 2: Dinamik Çağrı Akışının Parçası Olarak Kullanma (Pipeline)

Arama yaparken müşterinin adına özel dinamik ses üretip sonraki düğüme (Baresip'e) iletmek için:

1. Python Function Node (Dinamik Metin & Dosya Adı Üretici):

2. Python Kodu: ``python import time

```
# Gelen nesneden (dict) müşteri adını ve çağrı ID'sini al payload_data = msg.get('payload', {})  
customer_name = payload_data.get('name', 'Ahmet Bey') call_id = payload_data.get('call_id',  
int(time.time())) filename = f"call_{call_id}.wav"  
  
# HTTP Request node'unun beklediği JSON nesnesini ayarla msg['payload'] = { "text":  
f"Merhaba Sayın {customer_name}, ödemeniz başarıyla alındı.", "model": "tr_TR-eren-  
medium", "filename": filename } return msg  
2. ``HTTP Request Node``: ** - **Method:**  
`POST` - **URL:** `http://127.0.0.1:5000/api/tts` - **Return:** `a parsed  
JSON object` 3. ``Python Function Node`` (Çıktıyı Baresip İçin Hazırlama): **  
- **Python Kodu:** python # Piper API yanıtından dosya yolunu çek ve sonraki adıma ilet  
file_info = msg.get('payload', {}) msg['audio_path'] = file_info.get('file_path') # "/tmp/media/  
call_123.wav" msg['phone_number'] = "05321234567" return msg ``
```

2. FFMPEG (Medya Formatlama & Dönüştürme)

A. Arka Planda Ne Oluyor? (Low-Level Mekanizma)

Piper varsayılan olarak 22050Hz Mono WAV üretir. Telekom ağlarında (SIP/RTP) ideal ses kalitesi ve uyumluluk için ses dosyasının 8000Hz Mono PCM (G.711) formatına dönüştürülmesi gerekir.

• Arka Plan Komutu (CLI):

```
bash ffmpeg -y -i /tmp/media/welcome.wav -ar 8000 -ac 1 -c:a pcm_s16le /tmp/  
media/welcome_telecom.wav
```

B. Node-RED Python Yapılandırması

Senaryo 1: Bağımsız Dosya Formatlama

1. Python Function Node (FFmpeg Komutu Hazırlayıcı):

2. Python Kodu: ``python input_file = "/tmp/media/welcome.wav" output_file = "/tmp/media/ welcome_formatted.wav"

```
# Exec node'unun çalıştıracığı shell komutunu oluştur msg['payload'] = f"ffmpeg -y -i  
{input_file} -ar 8000 -ac 1 -c:a pcm_s16le {output_file}" return msg  
2. ``Exec Node``  
(Shell Komutu Çalıştırıcı): ** - **Command:** (Boş bırakılır, komut
```

`msg.payload` 'dan çekilir) 3. **Debug Node** : - Output: `msg.payload` (FFmpeg stdout çıktısını basar).

Senaryo 2: Akış İçinde Otomatik Formatlama (Pipeline)

1. **Python Function Node (Piper Sonrası Komut Üretici):**
2. **Python Kodu:**

```
python input_path = msg['payload']['file_path'] # "/tmp/media/call_123.wav"
output_path = input_path.replace('.wav', '_telecom.wav')

msg['formatted_audio_path'] = output_path msg['payload'] = f"ffmpeg -y -i {input_path} -ar
8000 -ac 1 -c:a pcm_s16le {output_path}" return msg 2. Exec Node: 

```
Command: `` 3. Python Function Node (Formatlanan Dosyayı Baresip'e
Aktarma):

```
**Python Kodu:** python msg['audio_to_play'] =
msg['formatted_audio_path'] return msg``
```


```


```

3. BARESIP (SIP Çağrı Motoru & ctrl_tcp)

A. Arka Planda Ne Oluyor? (Low-Level Mekanizma)

Baresip container'ı arka planda `ctrl_tcp.so` modülü ile `5555` portunda bir TCP Server çalıştırır.

- **TCP Sinyalleşme ve Komutlar:**
- **Arama Başlat:** `/dial sip:05321234567@sip-provider.com\n`
- **Ses Oynat:** `/play /tmp/media/welcome_formatted.wav\n`
- **Aramayı Kapat:** `/hangup\n`

B. Node-RED Python Yapılandırması

1. Arama Başlatma ve Ses Dinletme Akışı

1. **Python Function Node (Baresip Komutu Hazırlama):**
2. **Python Kodu:**

```
python target_number = msg.get('phone_number',
'sip:test@127.0.0.1') # Baresip TCP soketine gönderilen komutların sonunda \n
olmalıdır! msg['payload'] = f"/dial {target_number}\n" return msg
```
3. **TCP Request Node (Baresip ctrl_tcp bağlantısı):**
4. **Server:** `baresip`
5. **Port:** `5555`
6. **Return:** `String`

2. DTMF Tuşlamalarını Dinleme (IVR Menüsü)

1. **TCP In Node (Sürekli Dinleyici):**
2. **Server:** `baresip`

3. **Port:** 5555

4. **Output:** single String

5. **Python Function Node (DTMF Parser / Tuş Yakalayıcı):**

6. **Python Kodu:** ```python import json

```
line = str(msg.get('payload', '')).strip()
```

```
try: event = json.loads(line) if event.get('event') and event.get('type') == 'CALL_DTMF_START':  
msg['pressed_key'] = str(event.get('param')) # "1", "2", "3" return msg except Exception: if  
'dtmf:' in line: msg['pressed_key'] = line.split('dtmf:')[1].strip() return msg
```

```
return None # Tuşlama dışındaki olayları filtrele `` 3. ** Switch Node (IVR
```

```
Dallanması):** - Property: msg.pressed_key - Rule 1: == 1 -> (Satış Menüsü) -  
Rule 2: == 2` -> (Destek Menüsü)
```

4. TAM BİR IVR AKIŞ ÖRNEĞİ (Python ile Bağlantı)

[1. Inject: Arama Başlat]

|

▼

[2. Python Function: Metin & Parametre Hazırla]

|

▼

[3. HTTP Request: Piper TTS] (WAV Üretilir)

|

▼

[4. Exec Node: FFmpeg] (WAV -> 8kHz PCM'e Dönüştürülür)

|

▼

[5. Python Function: /dial Komut Cümlesi Oluştur]

|

▼

[6. TCP Request: Baresip 5555] (Telefon Çalar)

|

▼

[7. TCP In: DTMF Listener] → [8. Python Function: DTMF Parse] → [9. Switch: Tuş Kontrolü]

💡 İpuçları ve Python Kuralları

1. **Dictionary Sözdizimi:**

2. JavaScript'teki `msg.payload.name` yerine Python'da `msg['payload']['name']` veya güvenli erişim için `msg.get('payload', {}).get('name')` kullanılır.
3. **String Formatlama:**
4. Değişken birleştirirken Python `f"..."` (f-string) yapısını kullanmak çok pratik ve okunaklıdır.
5. **Filtreleme (`return None`):**
6. Bir mesajı sonraki düğüme iletmek istemiyorsanız Python kodunuzun sonunda `return None` döndürmeniz yeterlidir.
7. **Pip İle Anında Ekstra Python Paketi Yükleme:**
8. Node-RED içindeki Python ortamına canlı ortamda anında yeni bir paket (örneğin `requests`, `psycopg2-binary`, `redis` vb.) kurmak isterseniz terminalden şu komutu çalıştırmanız yeterlidir: `bash docker compose exec -u root node-red pip install requests`